

AIGC检测报告

报告编号: APIN-20260514-53642886

察验时间: 2026-05-14 16:35:04

【题名】 面向深度学习系统模糊测试的测试用例智能管理与分析

作者: jj

原文总字符数: 35244

送检字符数: 35244

识别参考文献: 0 篇

检测结果 疑似AI生成文本字符数: 8642

总

疑似AI生成文本总占比
24.52%

高

显著疑似AI生成文本占比
24.52%

中

一般疑似AI生成文本占比
0.00%

低

不标识
75.48%

疑似AI生成片段分布图

显著疑似(可能性>85%) 8642

一般疑似(50%<可能性≤85%) 0

不标识(可能性≤50%)

原文标注

正文

AI生成可能性

摘

要。深度学习框架（如PyTorch、TensorFlow）已成为人工智能应用中的基础软件，其底层算子实现的可靠性会直接影响上层模型训练与推理结果。模糊测试（Fuzzing）可用于发现这类框架中的底层缺陷，但在实际测试中仍存在用例冗余、崩溃日志难以分析以及生成过程缺少覆盖反馈等问题：大量相似或低收益用例会增加执行开销，非结构化长堆栈需要人工结合算子语义和运行环境判断根因，局部参数空间内的重复变异也会降低测试推进效率。本文设计并实现了一套结合语义检索与大语言模型（LLM）的模糊测试辅助分析平台。平台将测试用例、覆盖信息、崩溃日志和语义检索结果组织到同一分析流程中，用于支持用例筛选、故障定位以及报告和补盲脚本生成。CEI综合考虑单个测试用例的分支覆盖贡献和加权计算成本，通过动态百分位阈值筛选低收益用例。在2000例模拟实验中，该方法

在覆盖率损失不足4%的情况下将执行耗时缩减60%；在1000例真实物理实验中，Gcov/LCOV探针采集的物理命中行数仅下降0.005%，总执行耗时缩减34.5%。针对崩溃根因定位，系统实现了双空间Daikon边界引擎，即结合语义相似用例召回和参数约束推断来寻找可能触发崩溃的边界条件。该引擎召回与目标崩溃用例拓扑相同的健康组与崩溃组样本，将嵌套张量参数展平为一维数值键值对，并在多模板扫描框架下执行不变量推断。只有当某一数学约束在健康组样本中全部满足、在崩溃组样本中全部不满足时，系统才将其标记为可能导致崩溃的约束边界。实验结果显示，该引擎在五条已知约束上的召回率为100%，并能在真实数据中发现跨算子的隐式整除约束等非平凡边界条件，为崩溃原因分析提供了比堆栈日志更具体的参数依据。平台进一步引入检索增强生成（Retrieval-Augmented Generation, RAG）方法，将相似用例上下文和Daikon推断出的约束边界写入结构化四板块提示词模板，由LLM生成包含相关用例、可疑约束和复现线索的诊断报告，同时生成靶向补盲测试脚本。

与直接使用LLM输出相比，RAG辅助诊断的准确性评分提高50%，幻觉程度降低61%。此外，系统实现了基于multiprocessing.spawn的进程级隔离执行引擎，通过捕获子进程退出状态码拦截底层C++段错误与断言失败，避免单个异常用例导致主服务中断。深度学习框架的工程地位与安全隐患深度学习（Deep Learning, DL）框架已经成为人工智能应用开发中的重要软件层。以PyTorch和TensorFlow为代表的主流框架，向下封装异构计算硬件、内存分配和算子调度细节，向上提供数学算子（Operator）库、自动微分接口和模型构建工具。这种封装方式使开发者能够在较高抽象层次上构建和训练神经网络模型，但也使框架本身成为由Python前端、C++运行时、算子内核、自动微分模块和硬件后端共同组成的复杂系统。当系统规模扩大后，底层算子实现、数据类型转换、边界维度处理和内存管理中的缺陷，往往难以通过少量人工样例或常规功能测试暴露。与传统应用软件中较容易观察到的异常中断不同，DL框架中的部分缺陷具有较强隐蔽性。例如，在特定张量维度组合、广播规则或混合精度计算条件下，底层算子可能在正向传播或梯度计算过程中产生细小的数值偏差。这类偏差不一定触发运行时错误，也不一定在单次推理或训练日志中表现为明显异常。上层神经网络如果在包含此类缺陷的框架上完成训练，可能会将偏差吸收到权重更新过程中，从而影响最终模型行为。在自动驾驶感知、医疗影像分析等对输出稳定性要求较高的应用中，这类问题可能进一步表现为漏检、误判或结果不一致。除数值错误外，框架底层的内存越界、缓冲区溢出和断言失败也会带来运行稳定性风险。测试者或攻击者可以通过构造包含异常张量形状、特殊数据类型和边界参数组合的计算图（Computation Graph）触发非法内存访问，进而影响云端训练或推理服务的可靠性。对于同一个算子，不同输入维度、数据类型、设备后端和参数取值可能对应不同的底层执行路径。

86.71%
(735字符)

尤其是在算子组合形成复杂计算图后，单个算子内部的边界行为还可能与前后算子的输出形状或类型约束发生关联。这使得框架测试需要覆盖大量非常规输入，包括极端维度、空张量、非连续内存布局、异常广播关系和跨算子的参数约束。仅依赖人工经验构造测试样例，很难持续覆盖这些组合空间。模糊测试的引入与测试数据管理瓶颈针对深度学习框架中由复杂输入空间引发的缺陷，传统基于人工编写固定输入的单元测试方法存在覆盖不足的问题。框架接口需要同时处理多种数据类型、张量维度、设备后端以及算子之间的嵌套组合，输入空间很难通过少量手工样例完整枚举。因此，学术界与工业界逐渐将模糊测试（Fuzzing）引入DL框架测试场景。模糊测试通过自动化算法生成包含畸形参数、极端边界值或特殊算子组合的输入，并将其持续送入待测框架执行，以发现普通测试样例不易触发的崩溃、断言失败、数值异常或后端不一致问题。这些脚本中包含随机生成或变异得到的

算子拓扑、张量形状、数据类型、设备配置和属性参数。测试执行后，系统还会同步产生覆盖率数据、崩溃日志、执行耗时、异常类型、退出状态码和运行环境记录等产物。覆盖率数据记录测试用例触达的代码分支或物理行；崩溃日志可能混合Python调用栈、C++后端错误、底层断言信息和部分硬件相关输出；运行记录则用于说明该用例的执行时间、资源消耗和是否触发异常。这些数据共同构成后续分析的基础，但它们格式差异较大，来源也分散在脚本文件、日志文本、覆盖率报告和数据库记录中。当测试任务长时间运行或并发执行时，这些测试产物会迅速积累。如果缺少统一的数据组织方式，后续的用例筛选、崩溃归并和根因分析都需要较多人工参与。具体而言，测试脚本需要与其覆盖率记录、执行结果和崩溃日志建立对应关系；同一底层缺陷触发的多条崩溃日志需要被归并；覆盖收益较低但执行成本较高的用例需要被识别；历史失效样本中包含的参数边界和算子组合也需要被重新利用。

因此，DL框架模糊测试的难点并不只在于生成更多输入，还在于如何管理和分析生成之后的大量测试产物。首先，崩溃日志的检索和归类并不适合完全依赖关键字匹配。同一底层缺陷可能因输入脚本、变量名、张量形状或调用路径不同而产生不同表层日志；相反，日志中相似的错误词也可能对应不同算子、不同参数组合或不同后端问题。传统文本检索工具主要根据字面特征匹配，难以同时利用算子拓扑、参数结构和历史相似用例等上下文信息。此外，DL框架的底层计算引擎封装了大量调度细节，异常堆栈往往停留在运行时或C++内核层，难以直接映射到应用层触发该异常的具体算子组合。这会使开发者需要在测试脚本、日志文件和框架源码之间反复切换，从而拉长根因定位和缺陷复现的周期。其次，测试用例的多样性需要结合计算图语义来观察。模糊测试的效果依赖于生成算法对输入空间的探索广度，但变异引擎在实际运行中容易反复生成结构相近的算子组合。例如，不同脚本在文本上可能存在较大差异，但其计算图拓扑、张量维度变化和参数约束实际上高度相似；也可能出现脚本文本相近，但由于数据类型或边界参数不同而触发不同执行路径的情况。对于包含节点属性、边连接关系和多维张量配置的计算图，简单统计算子数量、脚本长度或异常次数难以反映测试集的实际分布。传统关系型数据库可以保存元数据，但不擅长表达高维结构之间的相似关系；基础统计图表也难以展示测试集在语义空间中的聚集、重复和空缺区域。因此，测试数据管理需要引入语义表示和相似检索机制，以支持对历史用例的召回、归类和后续分析。最后，单独观察覆盖率也不足以衡量测试用例价值。在模糊测试中，一个用例即使带来少量新覆盖，也可能消耗较长执行时间或占用较多显存；另一些用例虽然没有触发崩溃，但可能覆盖了尚未充分探索的算子组合。如果只使用代码覆盖率或是否异常等单一指标，系统难以判断某个用例相对于其执行成本是否值得保留。同时，覆盖率本身也不能说明一个用例是否与已有用例高度重复，更不能直接指出该用例是否有助于解释某类崩溃。

因此，需要同时考虑覆盖贡献、运行成本和语义差异，对低收益冗余用例进行筛选，并从历史崩溃样本中提取可复用的约束信息和补盲方向。深度学习框架模糊测试方法研究面向DL框架的模糊测试研究首先关注如何生成更容易触发异常的输入。Pei等人提出的DeepXplore引入了神经元覆盖率的概念，通过差分测试策略自动生成能够触发系统内部逻辑差异的测试输入，为后续深度学习系统测试提供了参考思路。Xie等人提出的DeepHunter进一步将变异策略从像素级扰动扩展到覆盖率引导的多层次变异机制，使测试生成过程能够根据覆盖反馈调整输入变异方向。这类研究主要面向深度学习模型本身，关注输入样本如何触发模型行为差异或覆盖更多神经元状态。这类方法的优势在于能够利用真实代码中的调用习惯，以较低人工成本覆盖大量框架接口。不过，从本文关注的问题来看

86.64%
(762字符)

，上述研究的重点仍然集中在生成端。它们主要回答如何产生更多测试输入、如何提高生成输入的覆盖能力，以及如何利用已有代码或LLM降低测试脚本构造成本。当测试工具已经持续产生大量脚本、覆盖率报告和崩溃日志后，如何组织这些产物、如何归并相似崩溃、如何筛除低收益用例、以及如何把历史失败样本转化为新的补盲方向，仍需要额外的系统支持。本文的工作因此更侧重生成之后的测试产物管理与分析：包括脚本、覆盖率、崩溃日志和语义向量的统一存储，以及基于历史测试结果的用例筛选、故障辅助定位和候选补盲脚本生成。测试数据管理与效能评估相关研究测试用例管理领域已有较多关于优先级排序和用例约简的研究。Rothermel等人提出了基于覆盖率的贪心排序策略，通过优先执行能够覆盖更多新代码分支的用例来提升回归测试效率。Harrold等人研究了在维持覆盖率约束下的最小用例集选择问题。这类方法在传统软件测试中应用较多，其基本假设是覆盖率向量能够较好地表示测试用例之间的差异，并且测试输入本身可以用相对明确的结构化特征描述。

在回归测试场景中，这一思路能够减少重复执行带来的成本。DL框架模糊测试产物与传统回归测试用例存在明显差别。一方面，测试输入不只是标量参数或简单对象，而是由算子拓扑、张量形状、数据类型和属性约束共同构成的高维结构。另一方面，测试执行结果也不只包括通过或失败，还包括覆盖率变化、崩溃堆栈、执行耗时和后端环境差异。如果仅根据文本相似度或覆盖率向量判断用例价值，容易忽略计算图结构和参数组合上的语义接近关系。例如，两个用例可能覆盖相近代码区域，但一个用例执行成本很低，另一个用例需要较长时间和更多显存；也可能出现两个用例覆盖率差异不大，但其中一个触发了更接近历史崩溃样本的参数边界。因此，用例约简和效能评估需要引入更适合高维测试数据的表示方式，并同时考虑覆盖贡献与计算成本。以Milvus为代表的向量检索引擎支持对稠密浮点向量执行近似最近邻（ApproximateNearestNeighbor,ANN）查询，能够在较短响应时间内完成大规模向量相似度匹配。将语义编码和向量检索用于测试数据管理，可以弥补关键字检索在处理崩溃日志和计算图脚本时的不足。通过向量化表示，系统可以根据语义相似性召回历史脚本和崩溃样本，而不仅仅依赖日志中的固定词项或脚本中的字符串片段。不过，单独使用向量检索并不能完成覆盖率分析、用例成本评估或约束边界判断。向量检索更适合回答哪些历史样本与当前样本相似，而不能直接判断该样本是否值得继续执行，也不能说明某个崩溃是否由特定参数边界触发。因此，本文采用MySQL与Milvus协同的存储方式，将结构化元数据、覆盖率记录和高维语义向量关联起来。在此基础上，系统进一步构建CEI评估模型，把覆盖贡献和执行成本放在同一评价框架下；同时结合相似用例召回和Daikon约束分析，为崩溃原因分析提供参数层面的辅助依据。

大语言模型辅助软件测试研究现状LLM在软件工程领域的应用正受到广泛关注。

在代码生成方面，Codex与CodeLlama等模型展示了根据自然语言描述生成程序代码的能力。在缺陷修复方面，多项研究表明LLM能够基于错误上下文生成候选修复补丁。这些工作说明，LLM可以参与测试脚本生成、错误解释和候选修复建议等环节。但是，通用LLM在处理DL框架故障诊断时仍存在限制。框架崩溃日志通常包含跨语言调用栈、底层运行时错误和版本相关行为。如果模型无法获得当前项目中的相似用例、参数配置、覆盖信息和历史执行结果，就容易根据通用经验给出并不适用的解释。这种现象在垂直测试场景中表现为幻觉（Hallucination）：例如虚构底层调度逻辑、误判触发算子，或推荐已经不适用于当前框架版本的参数配置。因此，在故障诊断任务中直接让LLM解释堆栈日志，往往难以保证输出与实际测试环境一致。检索增强生成（Retrieval-AugmentedGeneration,RAG）架构通过在模型推理前召回外部知识库中的相关上下文，可以在一定程度上约束模型输出。与直接调用LLM相比，RAG更适合将历史相似崩溃、覆盖信息、参数配置和约束分析结果提供给模型。现有研究更多关注LLM生成测试用例或生成修

86.62%
(742字符)

复建议，而本文系统将RAG与语义检索、双空间Daikon约束分析和补盲脚本生成结合。其中，语义检索用于召回与目标崩溃相关的历史样本；Daikon约束分析用于从健康样本和崩溃样本之间寻找可能的参数边界；RAG则负责把相似用例、可疑约束和复现相关信息组织为LLM可使用的上下文。这种设计使LLM不再只依据通用知识解释日志，而是结合系统已有的测试数据生成诊断报告和候选补盲测试脚本。研究目标与主要工作本课题围绕DL框架模糊测试产物的管理、分析和再利用展开。系统面向测试过程中产生的脚本、覆盖率数据、崩溃日志和运行记录，设计一套结合结构化存储、语义检索、效能评估和LLM辅助诊断的分析平台。研究目标不是替代现有模糊测试生成器，而是在测试输入产生之后，对测试产物进行组织、检索、筛选和诊断，从而提高历史测试数据的复用程度。

具体研究目标包括以下三个方面。第一，构建面向测试产物的异构数据管理架构。针对测试脚本、异常日志、覆盖率记录和运行元数据格式不一致的问题，系统使用MySQL保存结构化信息，使用Milvus管理由脚本和日志编码得到的高维语义向量。该架构用于建立测试用例、崩溃记录、覆盖数据和相似检索结果之间的关联，为后续缺陷归类和历史样本召回提供数据基础。在该目标下，系统需要完成日志解析、字段抽取、向量生成、索引构建和数据同步等流程，使原本分散在文件系统和日志目录中的测试产物能够被统一查询和分析。第二，建立测试用例多样性观察与效能量化机制。系统利用降维映射算法将高维测试用例特征投射到低维平面，辅助观察测试集在语义空间中的聚集和空缺情况。同时，构建覆盖效能指数（CoverageEfficiencyIndex,CEI）评估模型，将单个用例的覆盖贡献与执行成本结合起来，用于识别覆盖收益较低但执行开销较高的冗余用例。该目标对应系统中的多样性可视化模块和CEI筛选模块。与只统计覆盖率不同，CEI模型关注用例在覆盖收益和运行代价之间的关系，从而为后续测试集压缩和执行资源分配提供依据。第三，实现面向崩溃分析的语义检索、约束推断与RAG辅助诊断流程。系统通过语义检索召回与目标崩溃用例相似的历史样本，并利用双空间Daikon边界定界算法，结合语义相似用例召回和参数约束推断，分析可能触发崩溃的边界条件。在此基础上，系统将相似用例上下文、可疑约束和复现相关信息写入RAG提示词模板，由LLM生成诊断报告和候选补盲测试脚本。该流程不替代人工判断，而是为开发者提供更集中的参数依据和复现参考，减少从长堆栈日志中手工筛选有效信息的工作量。本文的主要工作包括：设计并实现自动化日志解析与双路向量化数据流水线；部署MySQL与Milvus协同的异构混合存储底座；提出并实现双空间Daikon故障边界定界算法；集成基于RAG架构的诊断助手与算子缺口补盲模块；在插桩编译的PyTorch环境中完成千例规模的物理对照实验，对CEI筛选、边界条件分析和系统性能进行验证。

上述工作对应系统的数据接入、存储检索、用例评估、故障分析和辅助生成等模块，构成DL框架模糊测试产物管理与分析流程的主要实现内容。本文共分为六章，各章内容安排如下。本章说明DL框架测试面临的输入空间复杂、测试产物分散和崩溃日志难以分析等问题，梳理国内外在DL框架模糊测试、测试数据管理与LLM辅助软件测试方面的相关研究，并给出本文的研究目标和主要工作。第二章介绍相关技术与理论基础。本章围绕后续系统设计所需的基础方法展开，包括DL框架结构、模糊测试技术、语义编码与向量检索方法、降维分析与不变量推断理论、RAG架构以及代码覆盖率采集技术。这些内容为后续章节中的存储设计、相似检索、CEI计算和Daikon约束分析提供理论基础。第三章进行系统需求分析与总体架构设计。本章明确系统的功能性与非功能性需求，分析测试产物管理、语义检索、效能评估和辅助诊断等功能场景，并说明前后端分离的混合后端架构、异构双库存储方

案和功能模块划分。第四章详细描述系统的核心算法与模块设计。本章包括日志解析引擎、DFS故障辅助定位模块、双空间Daikon边界定义引擎、CEI效能评估模型、RAG诊断助手与补盲机制。其中重点说明各模块的数据输入、处理流程和输出结果，以及它们在系统分析链路中的衔接方式。第五章展示系统的工程实现与功能界面。本章介绍后端服务的关键接口实现、数据库表结构设计、各功能页面的交互逻辑以及底层物理执行沙箱的构建过程。通过系统界面和功能流程说明平台如何支持测试数据导入、可视化分析、用例筛选和诊断报告生成。第六章设计并执行验证实验。本章通过模拟实验与真实物理对照实验评估CEI筛选算法的效果，验证Daikon边界定义引擎的准确性，并对系统整体性能指标进行分析。实验部分用于说明本文方法在用例筛选、崩溃分析和系统交互性能方面的实际表现。最后为总结与展望。该部分回顾本课题完成的主要工作，分析现有系统在数据规模、模型依赖和自动化程度方面的不足，并讨论后续可进一步改进的方向。

相关技术与理论基础深度学习框架与模糊测试技术深度学习框架的体系结构当前主流DL框架普遍采用分层架构设计。以PyTorch为例，其体系结构自顶向下可划分为三个层次。最上层是面向用户的Python前端接口，提供张量操作、自动微分以及神经网络模块的高级抽象。中间层负责组织用户定义的运算关系，在执行、追踪或优化过程中可形成计算图或中间表示（IntermediateRepresentation）。为便于后续分析，本文将算子之间的数据依赖抽象为有向无环图（DirectedAcyclicGraph,DAG）结构，其中节点对应算子调用，有向边描述数据的流向依赖关系。最底层是C++实现的ATen张量计算库与CUDA核函数调度引擎，直接处理GPU显存分配、并行线程调度与底层数值运算。这种三层结构使框架接口保持较高的易用性，也保证了底层执行效率。但从测试和故障分析角度看，分层封装会拉开用户脚本与底层算子实现之间的距离。如果异常还涉及计算图中前后算子的形状传递、数据类型转换或设备切换，仅依靠日志文本进行人工判断会更加困难。本文后续设计的故障辅助定位模块和语义检索流程，正是围绕这种跨层封装带来的分析困难展开。模糊测试技术原理与分类Fuzzing通过自动化方式持续生成测试输入，将其送入待测程序执行，并观察程序在运行过程中是否出现异常信号。依据输入的构造策略，Fuzzing通常可分为两类：基于变异（Mutation-based）的方法从已有种子输入出发，通过随机翻转、插入或删除字节来派生新的测试用例；基于生成（Generation-based）的方法则依据输入格式的语法规则或语义模型从零构建符合特定约束的测试输入。一个用例通常包含算子类型、张量形状、数据类型、设备配置以及算子之间的拓扑连接关系。因此，测试系统除了要保存脚本本身，还需要记录执行结果、覆盖率变化、崩溃日志和运行环境等信息。

DL框架中的崩溃也不只包括段错误与未捕获异常。在差分测试场景中，不同后端或不同实现之间出现数值结果不一致，也可能反映底层算子实现存在问题。这类异常未必触发操作系统信号，需要结合执行结果和额外预言机制进行识别。本文系统面向的正是这类结构复杂、来源分散的Fuzzing产物，因此需要在数据摄入阶段对脚本、日志和覆盖率信息进行统一归档与特征提取。文本语义表征与向量检索技术基于孪生网络的预训练语义编码模型传统词袋模型或TF-IDF方法主要根据词项出现频率构造稀疏向量，能够表达文本中出现了哪些关键词，但难以处理词序、上下文语义以及同义表达等问题。对于DL框架崩溃日志而言，同一类底层异常可能因为变量名、张量形状或调用路径不同而呈现为不同文本；相反，包含相似错误词的日志也可能对应不同算子或不同后端问题。因此，本文系统需要一种能够表示句子级语义接近关系的编码方法，而不是只依赖字面匹配。Reimers等人提出的Sentence-BERT在预训练语言模型的基础上引入孪生网络（SiameseNetwork）结构，使句子级嵌入向量之间的距离具有可比较性。具体而言，两条输入文本分别经过共享参数的编码器后，各自

86.67%
(738字符)

产出一个固定维度的稠密浮点向量，余弦相似度即可直接反映两段文本的语义接近程度。这一特性适合用于本文系统中的两类表征：一类是崩溃日志和报错堆栈文本，用于召回历史相似缺陷；另一类是测试脚本中的结构信息，包括算子拓扑、参数配置和结构摘要等内容，用于比较不同测试用例在计算图语义上的接近程度。本课题选用all-MiniLM-L6-v2作为具体的编码器实例。该模型基于6层Transformer架构蒸馏而来，输出维度为384维，在保持一定语义区分度的同时具有较低推理延迟。在本系统中，该模型被部署于数据摄入流水线的向量化环节。每条测试用例入库前，系统会将日志文本和脚本结构信息编码为稠密语义向量，并将向量标识与结构化元数据关联保存。

这些向量为后续相似用例检索、崩溃归类和测试集多样性分析提供数值基础。高维向量近似最近邻检索当向量集合规模持续增长时，逐一计算查询向量与库中所有向量之间的距离会带来较高延迟。在本文系统中，语义检索需要在历史测试脚本和崩溃日志向量中快速召回相似样本，因此需要专门的向量检索引擎支撑在线查询。Milvus是一款面向海量向量数据的开源检索引擎，支持多种索引类型与距离度量方式。该索引首先通过聚类算法将向量空间划分为若干Voronoi单元，查询时仅在距离查询点最近的若干单元内执行精确扫描，从而将搜索范围从全集集合缩减至局部子集。这种索引方式适合本文系统中先召回候选相似用例，再结合结构化字段补全信息的查询流程。向量间的距离度量采用余弦相似度。对于两个d维向量a与b，其余弦相似度定义为：其中 a_i 与 b_i 分别为向量a和b的第i个分量。该度量对向量的模长不敏感，更关注方向一致性。在本文系统中，崩溃日志和脚本结构信息的文本长度并不固定，经Sentence-BERT编码后得到的向量更适合通过方向接近程度判断语义相似性。因此，余弦相似度被用于相似崩溃召回、相似脚本检索以及RAG上下文构造前的候选样本筛选。降维分析与不变量推断方法本文系统会为测试用例生成384维语义向量。这类高维向量适合机器检索，但不便于研究人员直接观察测试集在语义空间中的分布情况。因此，本文将PCA用于测试集多样性可视化，帮助观察Fuzzing过程是否集中在少数相似结构附近，或是否已经覆盖了较为分散的算子组合区域。随后对C执行特征值分解：其中 λ_k 为第k个特征值， v_k 为对应的单位特征向量。原始数据通过 $Y=XW$ 即可降阶至p维表示。本课题将384维的测试用例语义向量通过PCA降至2维平面坐标，用于生成测试集多样性的空间分布散点图。如果大量样本在二维平面中聚集于局部区域，说明当前测试集可能存在结构重复；如果样本分布较为分散，则说明测试输入在算子组合或参数结构上具有更高差异。

86.69%
(769字符)

该降维结果不直接替代覆盖率指标，而是作为CEI筛选和补盲方向判断之前的辅助观察依据。Daikon动态不变量检测框架若某一模板在全部已观测轨迹上均成立，则将其报告为一条候选不变量。本文借鉴Daikon的模板匹配和运行时取值分析思想，但使用场景与经典程序不变量检测有所不同。系统面对的不是单一程序的正常执行轨迹，而是DL框架Fuzzing产物中成功执行样本与崩溃触发样本两组对照数据。对于同一类算子或相似计算图结构，系统将张量形状、算子属性和关键参数整理为可比较的数值键值对，再检查哪些约束在健康样本中普遍成立，却在崩溃样本中被破坏。这些约束不能直接等同于缺陷根因，但可以作为分析崩溃触发条件的候选边界。第四章将进一步说明本文系统如何结合语义相似用例召回与Daikon式约束推断，构建双空间边界定义算法。检索增强生成技术当用户发起查询时，检索器首先从外部知识库中召回与查询语义相关的文档片段，随后将这些片段作为上下文拼接至LLM的输入提示词中，引导生成器在给定上下文范围内产出回答。在本文系统中，外部知识库并不是通用百科文本，而是历史测试脚本、崩溃日志、相似用例元数据以及Daikon推断得到的候选约束。这一架构相较于直接调用LLM具有两方面优势。其一，通过外部知识注入减轻模型对参数记忆的依赖，降低Hallucination的发生概率。其二，知识库可动态更新而无需重新训练模型权重，使系统能够适应持续增长的领域数据。对于DL框架

故障诊断而言，RAG的作用主要在于把当前崩溃用例与历史相似样本、候选参数边界和覆盖信息放入同一上下文中，使LLM的输出不只依赖通用编程经验。本课题在两个场景中集成RAG机制：一是故障辅助诊断——以当前崩溃用例的语义向量召回历史相似案例，并将相关用例、可疑约束和运行信息写入提示词，为LLM生成诊断报告提供项目内上下文；二是算子缺口补盲——将识别出的未覆盖或覆盖不足的算子组合信息注入提示词，由LLM生成候选测试脚本。

在这两个场景中，RAG并不替代确定性分析算法，而是把语义检索、约束分析和覆盖信息组织为更容易阅读的诊断文本或脚本草案。其具体集成方式将在第四章展开。代码覆盖率采集技术代码覆盖率用于描述测试用例触达代码区域的情况，是后续计算用例覆盖贡献的重要数据来源。GNU工具链中的Gcov是面向C/C++程序的覆盖率采集工具。其工作原理分为三个阶段：在编译阶段，通过向GCC传递`-coverage`选项，编译器在源代码的每个基本块入口处插桩（Instrumentation）计数器指令，同时生成记录源码结构信息的`gcno`文件；在执行阶段，程序运行时各计数器累加实际命中次数，进程正常退出后将计数结果落盘为`gcda`文件；在分析阶段，Gcov读取`gcno`与`gcda`文件，计算并输出每个源文件的行级与分支级覆盖率。LCOV是对Gcov输出的上层封装工具。它支持对多次运行产生的覆盖率数据执行合并（Merge）与差集运算，并可生成HTML格式的可视化覆盖率报告。在本课题中，系统需要在插桩编译的PyTorch底层C++引擎上逐条执行测试用例并采集对应的覆盖率增量数据。LCOV的合并与差集能力使系统能够追踪单条用例对全局覆盖率的边际贡献。该数据不是单独用于评价测试充分性，而是与算子权重、算子出现次数等信息共同输入CEI模型，用于判断某条用例相对于其计算成本是否具有保留价值。本章围绕本文系统后续设计所需的技术基础展开说明。DL框架的分层结构解释了为什么崩溃日志难以直接对应到底层触发位置，Fuzzing技术则提供了持续生成测试脚本和暴露异常输入的基本方法。在测试产物进入系统后，Sentence-BERT负责将日志文本和脚本结构信息转化为可检索的语义向量，Milvus与ANN索引用于在历史样本中召回相似用例。PCA将高维向量投影到二维空间，为测试集多样性观察提供可视化基础；Daikon式不变量推断用于在健康样本和崩溃样本之间寻找候选约束边界。

86.69%
(707字符)

在分析结果需要呈现给用户时，RAG负责把相似用例、候选约束和运行记录组织为LLM可使用的上下文。Gcov与LCOV采集的覆盖率增量则为CEI计算提供数据来源。这些技术分别支撑后续章节中的数据存储、语义检索、多样性监测、边界分析、诊断报告生成和效能评估模块。系统需求分析与总体设计。根据第一章对DL框架Fuzzing产物管理问题的分析，本系统需要围绕测试数据接入、语义检索、效能评估和辅助诊断建立完整的数据处理流程。系统面向的输入主要包括Fuzzing工具生成的测试脚本、执行日志、覆盖贡献数据和运行状态记录，输出则包括结构化元数据、相似用例检索结果、多样性可视化结果、CEI评分、候选边界条件以及诊断报告。具体功能性需求包括以下六项。（1）自动化日志解析与特征提取。Fuzzing工具持续运行后会产生大量原始崩溃日志与测试脚本，其中既包含Python调用栈、C++后端错误和退出状态码，也包含算子名称、张量维度、设备配置和执行耗时等可结构化字段。系统需要在数据摄入阶段解析这些文件，提取错误类型、触发位置、张量维度配置和运行状态等元数据，并将日志文本和脚本结构信息编码为稠密语义向量。解析后的结构化字段用于列表筛选、统计和排序，语义向量则供后续相似用例检索、崩溃归类和多样性分析使用。（2）语义检索与故障辅助定位。用户在分析崩溃时，通常无法仅凭关键字确定某条日志是否与历史缺陷相同。因此，系统需要支持用户输入自然语言描述、错误片段或崩溃特征，在向量空间中召回语义接近的历史用例。检索结果应同时返回脚本摘要、错误类型、执行状态、相似度分数和相关元数据，便于用户比较当前崩溃与历史样本之间的

关系。在此基础上，系统还需要结合计算图拓扑关系和错误类型，给出可能相关的算子节点或参数位置，作为故障辅助定位的参考，而不是替代开发者完成确定性的根因判断。

(3) 测试集多样性监测。Fuzzing过程容易在局部参数空间或少数算子组合附近重复变异。

为了观察测试集是否出现明显聚集，系统需要将测试用例的高维语义特征降维到二维平面，并在前端以散点图形式展示。用户可以通过点的空间分布、状态颜色和异常类型筛选，观察当前测试集在语义空间中的聚集区域、稀疏区域和异常样本位置。该功能的输出不是单一数值指标，而是为后续用例筛选、补盲方向选择和测试集质量分析提供可视化依据。

(4) 效能评估与低价值用例清退。在长时间Fuzzing任务中，部分测试用例虽然会占用较多执行资源，但对分支覆盖率或边覆盖率的贡献较低。系统需要建立覆盖贡献与计算成本相结合的效能量化指标，读取覆盖贡献数据、算子权重和算子出现次数等信息，计算每条用例的CEI得分。用户可以查看单条用例的评分结果，也可以依据动态阈值标记低收益冗余用例。对于被确认清退的记录，系统需要在MySQL结构化记录和Milvus向量记录之间执行级联处理，降低单库残留数据对后续检索和统计造成干扰的风险。

(5) 故障边界界定。对于同一类算子或相似计算图结构，健康样本与崩溃样本之间往往存在参数取值、张量形状或算子属性上的差异。系统需要基于成功执行样本和崩溃触发样本构造对照组，将嵌套参数整理为可比较的数值键值对，并利用Daikon式约束推断方法寻找候选边界条件。这些候选约束可以帮助用户理解某类崩溃可能与哪些参数范围有关，但不应被直接表述为确定性根因。前端需要以可读形式展示候选约束、关联样本和相关参数，供开发者进一步确认。

(6) 智能诊断与算子缺口补盲。系统需要集成RAG机制，在崩溃诊断场景中将历史相似用例、候选约束、运行信息和当前崩溃上下文组织为提示词，为LLM生成诊断文本提供项目内数据依据。诊断结果应包含相关用例、可疑参数条件和复现参考等信息，帮助用户减少在长日志中手工筛选信息的成本。同时，系统需要根据当前测试集中未覆盖或覆盖不足的算子组合，构造补盲提示词并生成候选补盲测试脚本。这类脚本作为后续测试输入的候选材料纳入管理，仍需要经过执行验证和人工检查。

(1) 高并发写入性能。Fuzzing任务通常以批量或持续方式产生测试用例，在并发执行场景下，同一时间可能有多条脚本、日志和覆盖记录进入系统。如果摄入接口写入能力不足，测试结果会堆积在文件目录或消息队列中，影响后续检索和分析的时效性。因此，系统的数据摄入接口需要支持每秒数十条用例的并发写入，并在结构化元数据落盘和语义向量写入之间保持稳定的处理顺序。对于写入失败或部分失败的记录，系统需要提供重试或异常记录机制，降低数据丢失和双库状态不一致的风险。

(2) 大规模数据可视化性能。多样性监测页面需要在浏览器端展示万级规模的二维散点图。如果直接依赖普通DOM节点或未优化的Canvas绘制，在缩放、筛选和悬停查看详情时容易出现明显卡顿。因此，前端需要采用适合大规模点渲染的方案，在主流硬件配置下维持较为稳定的交互帧率。这一需求并不只是界面体验问题，还关系到研究人员能否在大量历史用例中快速观察聚集区域、异常样本和测试空缺。

(3) 底层执行隔离安全性。系统在采集覆盖率数据或验证候选测试脚本时，需要实际执行由Fuzzing产生的测试用例。这类脚本可能触发底层C++段错误、断言失败、死循环或资源占用异常。如果所有用例都在主服务进程内执行，单个异常样本就可能导致服务中断，并影响其他用户请求和正在进行的数据写入。因此，每条测试用例需要放入独立子进程中运行，主进程通过退出状态码和日志捕获获取执行结果，并可结合超时控制减少长时间挂起对服务的影响，从而隔离底层异常对Web服务本身的影响。

(4) 异构数

据一致性。本系统同时使用MySQL保存结构化元数据，使用Milvus保存高维语义向量。一次数据写入通常需要跨越两个不同存储系统，删除或清退操作也需要同时处理两边记录。由于两类数据库的事务机制和写入接口不同，系统难以简单依赖单库事务来覆盖完整流程。因此，设计中需要通过全局唯一标识符、写入状态记录、失败重试和级联删除策略维护两库关联，尽量降低MySQL中存在元数据但Milvus缺失向量，或Milvus中残留向量但MySQL记录已删除的风险。

系统总体架构设计。前后端分离与混合后端架构本系统采用前后端分离的整体架构，后端进一步拆分为业务后端与算法后端两个独立服务。这种设计并不是单纯出于框架划分，而是为了适配系统中两类差异较大的任务：一类是用户交互、分页查询、数据管理和权限控制等业务逻辑；另一类是语义编码、向量检索、PCA降维、Daikon边界分析和RAG上下文构造等算法任务。前端采用Vue3实现交互界面。系统界面包含测试用例表格、筛选表单、日志详情、散点图可视化、诊断报告展示等多类状态组件。同时，前端需要承载万级散点图渲染和交互筛选，因此在可视化部分采用大规模散点渲染方案，以降低大量DOM节点带来的渲染压力。业务后端采用JavaSpringBoot实现。该层主要负责接收前端请求、处理分页查询、维护关系型数据、执行权限校验，并对算法后端提供统一代理入口。测试用例元数据、错误类型、执行状态、CEI得分和时间范围等查询条件更适合通过关系型数据库完成组合筛选。因此，Java业务后端在系统中承担结构化数据管理和请求路由的职责。将这些计算任务封装为独立服务，可以避免在Java后端中引入复杂的跨语言模型调用逻辑，也便于后续调整算法实现。前端请求统一发送至Java业务后端的8080端口，涉及算法计算的请求由业务后端通过内部代理转发至Python算法后端的5000端口。这种代理转发模式对前端透明，可以降低前端同时维护多个服务地址和鉴权流程的复杂度。系统总体架构如图所示。异构双库存储架构本系统的数据资产包含两种不同形态：一类是结构化元数据字段，如case_uid、错误类型、算子名称、张量维度、时间戳、执行状态、覆盖记录和CEI得分等；另一类是由Sentence-BERT编码得到的384维稠密浮点语义向量，包括日志语义向量和脚本结构信息对应的向量标识。如果使用单一存储引擎管理这两类数据，要么难以高效完成多条件结构化查询，要么难以支持高维向量的相似度检索。

因此，系统采用MySQL与Milvus协同的异构双库存储架构。MySQL负责保存结构化元数据和业务查询所需字段。这些数据需要支持按状态、时间范围、错误类型、分数区间和算子名称进行筛选，也需要支持分页、排序和事务性更新。关系型数据库的索引机制和事务能力更适合处理此类操作。Milvus负责保存高维语义向量及其少量标量附属字段。日志文本和脚本结构信息经过Sentence-BERT编码后，形成固定维度的稠密向量。这些向量无法通过普通B+树索引表达语义距离，需要借助IVF_FLAT等向量索引进行近似最近邻检索。因此，Milvus主要用于根据查询向量召回相似日志、相似脚本或候选历史用例，而不承担复杂业务条件查询和事务性数据维护。两库之间通过全局唯一标识符case_uid建立关联。每条测试用例入库时，系统首先生成case_uid，并将其写入MySQL主记录。在向Milvus写入日志向量和结构相关向量时，同一case_uid也会作为向量记录的标量字段写入。查询时，系统根据请求类型选择主查询库：如果用户执行结构化筛选，业务后端先从MySQL查询满足条件的用例，再根据需要补充向量标识或检索结果；如果用户执行语义检索，算法后端先在Milvus中召回相似向量及其case_uid，再回到MySQL批量查询对应元数据。最终结果通过case_uid完成拼接后返回前端。这种设计能够在结构化查询和语义检索之间取得较好的功能分工，同时通过统一标识降低双库数据不一致带来的影响。系统技术栈选型说明基于上述架构决策，本系统各层技术栈选型如下。技术选型主要围绕系统已有的数据形态、算法依赖和交互需求展开，而

86.62%
(683字符)

不是简单比较不同框架的通用优劣。前端层采用Vue3框架配合TypeScript进行开发。本系统前端需要处理较多状态联动：包括测试用例筛选条件、分页结果、日志详情、散点图选中状态、诊断报告展示和清退确认流程。

TypeScript则用于约束接口返回数据结构，减少前后端字段变更带来的运行时错误。UI组件库选用ElementPlus，其表格、表单、对话框和分页组件能够覆盖管理系统的基础交互需求。大规模散点图的渲染依赖专门的图形渲染方案，以满足万级数据点的实时交互要求。业务后端层采用Java17与SpringBoot3框架。该层面向的是结构化业务数据和前端请求组织，需要处理分页查询、动态筛选、数据更新和异常响应。数据库连接池采用HikariCP，用于提升高并发访问下的连接复用效率。这一层同时作为前端与算法后端之间的代理，减少前端直接调用多个后端服务的复杂度。该层依赖的核心Python库包括：这些库与本文系统的算法模块直接对应：存储层中，MySQL8.0负责结构化元数据管理，Milvus2.3.x负责高维语义向量的索引与检索。两者均部署于本地服务器，通过各自的SDK与后端服务通信。MySQL更适合事务性记录和条件查询，Milvus更适合向量相似度召回。二者通过case_uid保持数据关联，共同支撑系统的检索、分析和展示功能。系统功能模块划分与数据流本系统的功能模块按照职责划分为六个子系统：数据摄入与解析模块、语义检索与故障辅助定位模块、多样性监测模块、效能评估与清退模块、边界界定模块以及智能诊断与补盲模块。其中，语义编码、向量检索、PCA降维、Daikon边界分析和RAG上下文构造等算法接口主要封装在Python算法后端；结构化查询、分页展示、清退确认、数据管理和前端接口组织则由Java业务后端承担。业务后端根据用户请求调用自身数据接口或转发至算法后端，再将处理结果统一组织后返回前端。这种划分使数据接入、检索分析、效能评估和诊断生成之间保持相对清晰的边界，也便于在后续实现中分别调试各类业务逻辑和算法逻辑。系统的端到端数据流转路径如下。原始Fuzzing产物经数据摄入接口进入系统，其中包括测试脚本、崩溃日志、执行状态、覆盖记录和运行环境信息。

日志解析模块首先对原始文本进行清洗，过滤冗余调用帧，提取错误类型、状态码、执行耗时、算子信息和张量参数等结构化字段。对于能够表示计算图结构的脚本信息，系统会进一步整理算子拓扑、连接关系和参数摘要，作为后续向量化和辅助定位的输入。解析结果随后进入向量化环节。日志文本会生成日志语义向量；脚本结构信息会生成结构相关向量或向量标识。结构化元数据写入MySQL，语义向量写入Milvus，两类记录通过case_uid保持关联。在写入过程中，系统需要记录两库写入状态，并在部分失败时执行重试或异常登记，以降低后续查询中出现孤立记录的风险。入库完成后，下游分析模块可按需启动。语义检索模块接收自然语言查询、错误片段或目标用例标识，在Milvus中召回相似向量，再通过case_uid从MySQL补全脚本摘要、错误类型、执行状态和覆盖记录等元数据后返回前端。故障辅助定位逻辑结合错误类型和计算图拓扑关系，输出可能相关的算子节点或参数位置。多样性监测模块定期或按用户请求读取测试用例向量，执行PCA降维并输出二维坐标，供前端散点图渲染。效能评估模块读取覆盖贡献数据、算子权重和算子出现次数，计算各用例的CEI得分，并根据动态阈值标记低收益冗余用例。当用户确认执行清退时，系统依据MySQL中保存的向量主键同时删除Milvus向量记录和MySQL元数据记录，降低双库残留不一致的风险。边界界定模块接收目标崩溃用例后，先召回相似健康样本和崩溃样本，再对参数键值对进行比较，输出候选边界条件和对应样本依据。智能诊断模块则整合语义检索结果、候选约束、运行信息和覆盖情况构建RAG上下文，调用LLM生成诊断报告或候选补盲脚本。数据在各模块间的流转路径如图所示。系统详细设计与核心算法

自动化日志解析与双路向量化数据流水线基于分层正则匹配的日志解析引擎Fuzzing工具产生的原始崩溃日志在格式上具有高度异构性。

同一条崩溃记录可能混杂Python层异常堆栈、C++底层段错误信号、CUDA运行时显存报错以及多线程交织产生的乱序输出。将此类非结构化文本转化为可供后续模块消费的标准化元数据，是整条数据流水线的入口环节。本系统设计的LogParser引擎采用分层正则匹配策略，将解析过程分解为错误分类与细粒度字段提取两个独立阶段。在错误分类阶段，引擎维护一张优先级有序的错误模式表，涵盖八种面向DL框架的标准化错误类别，如表所示。引擎将输入文本统一转换为小写后，依照表中自上而下的优先级逐条扫描关键词。一旦命中即终止扫描并返回对应类别标识。优先级排列遵循危害性递减原则：显存溢出与段错误属于系统级致命异常，排在最前；数值异常属于静默型偏差，排在最后。对于不命中任何模式的日志，引擎将其归入other兜底类别。该分类结果用于为后续故障辅助定位和检索分析提供错误标签，不直接等同于底层缺陷根因。在字段提取阶段，引擎针对每种数据类型设计了专用正则表达式族，具体提取的结构化字段包括以下六项：通过匹配Python异常类名模式（如（w+Error）：）提取。DL框架的维度报错存在三种文本格式。TensorFlow使用函数调用表示法size（32,512）；（3）框架源码位置（source_location）。引擎分别以两组正则提取文件名、行号与模块路径。仅在cuda_oom类别下激活。引擎提取三组数值：请求分配量、已分配量与设备总容量。通过全局扫描cuda:N与cpu标识符提取，去重后输出为列表。保留未经处理的完整日志文本，供RAG模块作为LLM上下文素材。面对多线程交织的脏日志，引擎在解析完整执行日志时采用逐行状态机策略。当检测到ERROR级别时，引擎暂存该行作为后续结构化解析的输入。不含有效级别标签的行视为前一条日志的续行或冗余调用帧，直接过滤。整条日志的最终输出包含事件时间线列表、最终状态枚举、总执行耗时与末尾错误的结构化解析结果。

基于DFS拓扑回溯的故障辅助定位模块日志解析完成后，系统需要将结构化错误信息映射回测试用例的计算图结构，定位出可能与崩溃相关的候选算子。本系统设计的FaultLocator模块基于逆向邻接矩阵与DFS回溯机制实现该映射，其输出用于辅助用户缩小排查范围，而不是直接给出确定性根因。逆向邻接矩阵的构建过程如下。在逆向邻接矩阵中， o_{i+1} 反向指向 o_i 。逆向矩阵的构建使得引擎能够从错误触发点出发，沿数据流反向回溯至上游相关算子。每个算子节点的出度（Degree）定义为该节点在逆向邻接矩阵中的后继数量。对于线性连接链，所有节点的出度均为1；对于包含残差连接或多分支结构的计算图，分支汇聚节点的出度大于1。DFS回溯在以下条件之一满足时终止：（a）当前节点属于错误相关算子集合；（b）回溯深度超过连接链总长度的二分之一；（c）当前节点出度为零（即已到达计算图输入端）。引擎针对六种错误类别设计了差异化的定位策略，各策略依据不同的算子能力标注集合筛选候选对象。以shape_mismatch错误为例，引擎在连接链中扫描所有属于维度变换集合的算子，初始置信度设为0.85。若日志中同时包含具体的张量维度信息且候选算子为Linear或Conv2d，则置信度提升至0.95。引擎还检查特殊拓扑模式：若Conv2d与Linear直接相邻而中间缺少Flatten层，则对Linear节点的置信度追加0.05（上限为1.0），因为这是DL框架中常见的维度不匹配场景。最终输出的候选算子列表按置信度降序排列，每个条目包含算子类型、在连接链中的索引位置、置信度数值与自然语言形式的推理理由。该列表将传递给前端在计算图可视化中高亮标注候选节点，供用户结合日志、源码和复现结果进一步确认。双路向量化特征表征策略如第二章所述，本系统采用Sentence-BERT编码模型将离散文本映射为384维稠密浮点向量。

86.60%
(692字符)

针对Fuzzing产物的异构特性，系统设计了两条并行的向量化路径。第一条路径为报错堆栈语义表征。该路径产出的向量存入Milvus的fuzzing_logs集合，用于支撑后续的语义检索功能。第二条路径为图拓扑序列投影。引擎将测试用例的模型结构字典序列化为标准化文本，该文本经Sentence-BERT编码后产出的向量存入Milvus的fuzzing_structure集合，用于支撑多样性分析与拓扑相似性召回。两条路径在同一批次内并行执行向量化运算，共享同一个预加载的all-MiniLM-L6-v2模型实例以避免重复加载开销。异构混合协同存储与生命周期管理结构化元数据与向量特征的隔离协同设计。如第三章总体架构所述，本系统采用MySQL与Milvus的异构双库存储方案。两库各自承载适合其存储模型的数据类型，通过MySQL中保存的case_uid和向量主键共同维护跨库关联。Milvus侧维护两个向量集合。两个集合采用相同的Schema配置：索引类型选用IVF_FLAT，相似度度量选用余弦距离（COSINE），倒排列表数量（nlist）设为128，检索时探测簇数（nprobe）设为16。该配置用于减少高维向量检索时的全量扫描开销。跨库绑定的具体机制为：数据摄入时，引擎将双路向量分别插入两个Milvus集合，Milvus向量记录本身仅包含自增主键id与向量字段embedding。检索时，系统通过Milvus返回的向量主键批量查询MySQL元数据，也可以通过MySQL中保存的向量主键定位对应向量记录。这种双向关联方式与第三章的异构双库存储设计保持一致。异步协程削峰与高并发双写一致性保障在批量Fuzzing场景下，测试执行引擎可能在短时间内产生大量崩溃记录。若每条记录都同步触发一次MySQL写入加两次Milvus写入，网络往返延迟将成为系统吞吐量的瓶颈。该机制的核心数据结构为一个容量上限为B的异步队列（默认B=256）。

当队列中的记录数量达到批量阈值T（默认T=64）时，消费者协程被唤醒并执行一次批量落盘操作。此外，系统还注册了一个周期为t（默认t=5秒）的定时器协程。即使队列未滿，定时器到期时也会将队列中所有积累的记录一次性刷入数据库。这种容量触发+时间触发的双重机制兼顾高并发场景下的吞吐量与低负载场景下的数据及时性，同时避免记录长时间滞留在内存中。当队列已滿时，生产者协程进入背压（Backpressure）等待状态。asyncio的Queue.put（）方法在队列容量已达上限时自动挂起当前协程，直到消费者取走部分记录后才恢复执行。该机制以协程级别的非阻塞挂起替代了传统的线程阻塞，避免了线程池资源的浪费。批量落盘操作内部的执行顺序遵守先Milvus后MySQL原则。原因在于Milvus的插入操作返回自动生成的向量主键，该主键需嵌入MySQL记录的parameters字段中。若MySQL先行写入则缺少向量主键信息，可能导致跨库关联不完整。两步操作之间不引入分布式事务机制，而是通过以下补偿策略维护最终一致性：若Milvus写入成功但MySQL写入失败，系统将失败记录重新推入队列尾部并记录告警日志；若连续三次重试仍失败，该批次记录被转移至死信队列等待人工介入。该机制用于降低双库数据不一致风险，不等同于严格意义上的分布式事务保证。基于CEI阈值的跨库级联清理机制随着Fuzzing持续运行，数据库中将积累大量低价值的冗余测试用例。如第三章所述，本系统通过CEI指标量化每个用例的覆盖效能贡献。当用户触发清退操作时，系统需要在MySQL与Milvus之间执行基于向量主键的级联删除流程。动态阈值的计算采用中位数策略。系统首先从MySQL中查询所有用例的CEI值并计算其中位数，低于（默认=0.5）的用例被标记为清退候选。中位数相较于均值对极端离群值具有更强的鲁棒性，避免了少数高CEI用例拉高阈值导致误删的风险。

级联删除流程分为三步。为减少脏读，删除操作期间系统对涉及的case_uid集合加入内存级互斥标记。检索接口在返回结果前过滤掉所有带有删除标记的用例，降低用户在清退过

程中看到即将被移除数据的风险。Milvus侧的删除操作需在执行后调用 `collection.flush()` 强制持久化，否则已删除的向量在Segment合并之前仍可能被检索命中。由于MySQL与Milvus不共享事务管理器，该机制属于级联清退与最终一致性维护，而不是严格意义上的分布式事务保证。双空间Daikon故障边界定界引擎本节阐述系统中的双空间Daikon故障边界定界模块。该引擎借鉴经典的Daikon动态不变量推断范式，在成功执行样本与崩溃触发样本构成的对照组中扫描候选约束，输出用于辅助分析崩溃触发条件的候选边界条件。对照组召回与参数展平机制定界引擎的输入为一个目标崩溃用例。引擎首先提取该用例的图拓扑结构向量，在Milvus的fuzzing_structure集合中执行余弦相似度检索，分别召回拓扑结构相似但执行成功的10个用例（健康组）与执行失败的10个用例（崩溃组）。召回条件为结构相似度大于0.85，确保对照样本与目标用例处于相近的算子组合语境下。召回完成后，引擎对每个用例的嵌套参数字典执行展平操作。DL测试用例的参数空间通常具有多层嵌套结构：顶层为模型结构字典，下辖各层配置子字典，每个子字典又包含标量参数与列表参数。展平操作的目标是将这种N层嵌套结构归约为一维的键值对集合。展平函数的递归映射规则为：展平过程中，引擎过滤以下无关字段：过滤后的键值对集合仅保留与算子配置直接相关的数值参数。展平完成后，引擎计算健康组与崩溃组所有样本的键集合交集：仅在中的键才进入后续规律探测环节，确保所有对照样本在相同的参数维度上具有可比性。候选约束扫描与边界条件推断本小节阐述引擎的核心判定逻辑。引擎在公共键集合上系统性地扫描三层模板池，逐层递增变量数量与公式复杂度。

仅当某条候选约束同时满足成功全票通过与失败全票否决的双重验证判据时，该约束才被加入候选边界条件集合。这些候选边界条件反映当前对照样本中的参数差异，不能直接等同于确定性根因。第一层：单变量约束扫描。对中的每个键 x ，引擎测试以下模板：（b）奇偶性模板。验证 $x \bmod 2 = 0$ 是否在健康组全部成立且在崩溃组全部违反。该模板对应部分算子的通道数或分组参数约束，例如DL框架中的GroupNorm、PixelShuffle等算子对通道数存在整除关系要求。第二层：双变量关系约束扫描。对中的任意两个键 x 和 y (xy)，引擎测试以下模板：（b）整除关系模板： $xy \bmod y = 0$ （要求 $y \neq 0$ ）。该模板对应卷积分组操作中的整除约束，例如in_channels能被groups整除。该模板覆盖了DL框架中常见的倍数关系约束，例如转置卷积的输出尺寸公式中的2倍因子。第三层：三变量约束扫描。对中的任意三个互不相同的键 x 、 y 、 z ，引擎测试以下模板：（a）空间坍缩公式： $x + 2yz$ 。（b）张量维度乘积守恒： $xy = z$ 。该模板对应Reshape操作的元素总量守恒约束——重塑前后张量的元素总数必须相等。上述三层扫描的双重验证判据以算法的伪代码形式给出。算法的时间复杂度分析如下。每个候选谓词需遍历N个样本进行验证。在实际场景中，展平后的公共键数量通常不超过30，对照组总样本数为20，因此单次定界的计算耗时在毫秒量级。提取出的约束集合经去重后按优先级排序：多变量约束排在单变量约束之前，因为多变量约束包含更丰富的变量间交互信息，对崩溃条件的解释力更强。排序后的约束列表即为该崩溃用例的候选边界条件集合，用于为开发者提供后续排查线索。约束解释与诊断文本生成候选边界条件以形式化公式表达，便于程序验证，但对非专家用户缺乏可读性。本系统将Daikon式约束推断结果与RAG诊断文本生成相结合，把形式化约束、目标崩溃用例和相似历史样本组织为LLM上下文。

约束推断阶段即上述Daikon定界引擎，输出形如的候选数学约束。诊断文本生成阶段基于RAG机制调用LLM。引擎将候选约束集合、目标崩溃用例的结构化元数据、以及召回的相似历史用例上下文组装为结构化提示词模板，注入LLM。提示词的核心指令要求LLM完成三项

任务：（1）将候选约束翻译为面向开发者的自然语言解读；（2）结合具体的参数数值解释违反约束的可能含义；（3）给出可供人工检查的参数修正建议或候选安全参数组合。LLM的输出经格式化后与约束推断结果并列展示于前端界面。用户可同时获得可机器验证的数学公式与可阅读的诊断文本草案。这种设计降低了单独依赖LLM可能产生的幻觉风险：候选约束由确定性算法从真实样本中提取，LLM主要负责解释和组织文本，其输出仍需结合源码和复现实验确认。在工程实现上，两部分采用串行模式：约束推断先行完成并输出约束集合，随后诊断文本生成阶段将作为上下文的一部分送入LLM。串行设计使生成内容围绕已有候选约束展开，减少LLM输出偏离实际样本数据的风险。测试集多样性度量与CEI效能评估模型基于PCA的高维特征降维与空间分布监测Fuzzing过程持续产出的测试用例在高维语义空间中的分布状态，可以反映测试集在算子组合和参数结构上的多样性。然而384维向量空间无法被直接观测，需要降维投影至二维平面供人类判读。本系统在批量更新后或用户请求多样性分析时，对当前样本集合统一执行PCA降维。当样本数不足2时，PCA无法提取有效主成分方向。此时系统退化为随机微扰策略，为每个样本生成幅度为0.01的随机坐标，避免前端散点图中所有点重叠于原点。当样本数恰好为2时，PCA仅能提取一个主成分，系统将第二维度填充为零后仍执行归一化操作。前端散点图基于上述二维坐标渲染。用户可观察测试集的空间分布形态：若散点呈现明显的簇状聚集，说明当前Fuzzing策略可能集中在有限的算子组合区域内；若散点分布较为分散，说明测试输入在模型结构或参数组合上具有更高差异。

稀疏区域对应尚未被充分测试的算子拓扑组合，可作为后续补盲策略的参考方向。覆盖效能指数模型CEI用于量化单个测试用例在覆盖贡献数据与计算成本之间的效费比。其中覆盖贡献数据在公式中具体表示为分支边覆盖数 E_{b} 。直觉是：一个仅包含两个轻量算子却触发了高边覆盖数的用例，其效能远高于一个包含十层复杂算子但仅覆盖少量分支的用例。CEI的数学定义如下：其中 E_{b} 为该用例的分支边覆盖数（由Gcov/LCOV采集）， m 为该用例计算图中不同算子类型的数量， c_i 为第 i 类算子在计算图中的出现次数， w_i 为第 i 类算子对应的标准化权重系数。公式分母表示基于算子类型与出现次数估计的计算成本，不包含执行耗时变量；执行耗时可作为实验评估或运行监控指标单独记录。算子权重 w_i 的设计原则为：计算密度越高、参数空间越大的算子赋予越大的权重，因为此类算子消耗更多的框架执行资源却不一定带来更高的覆盖贡献。表列出了本系统采用的部分算子权重基准。若该用例的边覆盖数为240，则 $\text{CEI} = 240 / 8.0 = 30.0$ 。系统在执行清退时，以全库CEI中位数的倍数为淘汰阈值。CEI低于该阈值的用例被视为高消耗低贡献候选样本，进入级联删除流程。该机制用于帮助识别低效冗余数据，减少其对存储空间和后续分析结果的干扰。算子组合缺口的关联分析机制除对单个用例的效能评估外，系统还需在全局视角下分析测试盲区——即DL框架支持但当前测试集尚未覆盖的算子组合模式。以PyTorch为例，涵盖卷积族、线性族、循环族、注意力族、归一化族、池化族、激活族与结构操作族共八大类约60种算子。具体构建方法为：遍历MySQL中所有测试用例的连接链，对于每条连接链 $o_0o_1o_k-1$ 中的每对相邻算子 (o_i, o_{i+1}) ，遍历完成后， A 中值为1的元素对应已覆盖的算子对，值为0的元素对应尚未被任何用例触及的算子对。盲区提取通过集合差运算实现。定义理论可行算子对全集 $\mathcal{A}$ 为标准算子库中所有在DL框架语义下合法的前后衔接组合（排除物理上不可能的对，例如Softmax不能直接接Conv2d的通道输入端）。

盲区集合定义为：盲区集合中的每个元素 (o_i, o_j) 代表一对尚未被测试的算子衔接。系统将按算子类别分组后输出为结构化列表，每个条目包含源算子类型、目标算子类型以及

该对组合在框架文档中的参数约束摘要。该列表将作为后续RAG补盲模块的输入靶标。基于RAG的辅助诊断与候选补盲机制环境感知辅助诊断当用户针对某一崩溃用例发起诊断请求时，系统启动环境感知辅助诊断流程。该流程由四个串行阶段构成。第一阶段为特征向量化。系统将用户查询文本经Sentence-BERT编码为384维向量。第二阶段为双路并行检索。第三阶段为动态权重融合。系统根据查询文本中包含的结构关键词与日志关键词比例，动态调节两路检索得分的融合权重：若查询同时包含结构关键词与日志关键词，双路权重各取0.5；若仅包含结构关键词，结构路权重提升至0.8；若仅包含日志关键词，日志路权重提升至0.8。融合后按得分降序截取前K个结果。第四阶段为上下文组装与LLM推理。系统从MySQL中反查召回用例的完整元数据，将case_uid、执行状态、边覆盖数、执行耗时、错误信息与算子列表拼装为结构化上下文文本，连同用户原始查询一并注入LLM。LLM基于检索到的真实历史案例生成诊断文本草案，用于提示相似用例、可疑参数条件和排查方向。该输出仍需要用户结合源码和复现实验进行检查。算子缺口靶向补盲与候选用例生成盲区集合标识了当前测试集的覆盖缺口。本系统通过构造结构化提示词，引导LLM生成候选补盲脚本草案以覆盖这些缺口。结构化提示词设计如下，由四个功能板块组成：上述四板块模板的工程意义在于：System板块限定LLM的角色边界，减少其偏离测试生成任务；Context板块注入真实历史数据作为参考范式，降低无上下文生成的风险；Constraint板块施加语法与结构约束，使输出更便于后续检查和执行验证。LLM生成的候选测试脚本可经过以下验证流程后再入库：（1）Python语法检查（ast.

parse），过滤不合法脚本；（2）模型结构提取，解析连接链并验证是否包含目标算子对；（3）轻量级试运行（可结合超时保护），验证脚本可正常执行或产出有效崩溃。通过验证的脚本以新测试用例身份进入数据摄入流水线，形成发现盲区→生成候选脚本→执行验证→入库分析的处理流程。本章围绕系统的核心算法层展开了详细的技术阐述。4.1节设计了分层正则匹配的日志解析引擎与基于逆向邻接矩阵的DFS故障辅助定位模块，实现了从非结构化崩溃日志到候选算子定位的处理链路。4.2节构建了MySQL与Milvus的异构协同存储方案，通过异步协程缓冲队列缓解高并发双写带来的吞吐压力，并通过补偿重试和级联删除降低双库数据不一致风险。4.3节提出了双空间Daikon定界引擎，以成功全票通过+失败全票否决双重验证判据，从对照组参数空间中提取候选边界条件，并结合RAG机制将形式化约束组织为可读诊断文本。4.4节建立了CEI效能评估量化模型与算子组合盲区分析机制，明确了覆盖贡献数据、算子权重和算子出现次数在效能计算中的作用，为后续候选补盲提供目标输入。4.5节完成了基于RAG的辅助诊断与候选补盲设计，通过四板块结构化提示词模板引导LLM生成诊断文本或候选补盲脚本，形成分析→定位→候选补盲→验证的处理流程。上述各模块协同构成了本系统从数据摄入、语义检索、边界定界到候选补盲的核心算法流程。系统实现与功能展示开发环境与工具配置本系统开发、测试与部署涉及前端、业务后端、算法后端与底层物理执行四个层面。表列出了完整的软硬件环境配置。当前实现中，各层之间的通信链路安排如下。分页查询、状态筛选、详情获取和统计概览指向Java后端（端口8080）；向量计算、语义检索、RAG推理、Daikon定界、CEI分析和补盲建议由Python后端（端口5000）处理。数据导入和若干分析页面则直接调用Python服务。底层物理执行环境部署于WSL2内的Ubuntu22.

04子系统。物理执行引擎通过multiprocessing模块的spawn启动方式创建隔离进程，在WSL环境下调用插桩后的PyTorch库执行测试脚本并采集覆盖贡献数据。后端核心模块实现服务初始化阶段建立Milvus连接、加载两个向量集合，并加载预训练的Sentence-BERT编

码模型；MySQL连接则在具体请求处理过程中按需建立。表列出了主要路由的方法、路径与功能摘要。该接口的请求体仅包含一个字段：其中failed_case_uid为目标崩溃用例的全局唯一标识。接口内部执行以下处理链路：（1）从MySQL查询目标用例的完整元数据与向量主键；（2）以向量主键在Milvus中提取结构特征向量；（3）在Milvus中执行余弦相似度检索，召回20个最近邻样本；（4）将近邻样本按执行状态分为健康组（最多10个）与崩溃组（最多10个）；（5）对健康组与崩溃组执行参数展平与张量Diff对比；（6）调用Daikon定界引擎提取候选约束；（7）根据候选约束构造满足约束与不满足约束的轻量样例，并通过PyTorch算子级执行结果判断约束是否具有参考价值；（8）将Diff结果、候选约束与验证报告共同注入LLM提示词，生成面向开发者的诊断文本草案。包含uid（崩溃用例标识符，字符串类型）与error（原始错误文本，字符串类型）两个子字段。包含uid（最近邻健康用例标识符）一个子字段。kernel_size），值为对应的数值。该字段供前端渲染参数对比视图。数组按变量数量降序排列，多变量约束排在前面。每个样例进一步包含被变异的参数路径、变异后的参数值、约束求值结果、实际执行状态以及报错摘要。若所有正样例均能成功执行，且所有反样例均触发运行时错误，则is_validated字段置为真，表示该约束在当前样本和轻量级验证样例上具有较高可信度。LLM基于上述上下文生成的自然语言诊断摘要。

提示词中包含daikon_validation的执行验证结论，使生成文本围绕候选边界条件展开，降低将偶然相关关系误解释为确定性根因的风险。每个TestCase对象包含以下字段：响应体包含success（成功摄入数量）、ids（结构向量主键列表）与message（状态描述）三个字段。该接口体现了第4章所述的双路向量化设计：服务根据model_structure构造结构描述文本，根据error_message或成功占位文本构造日志描述文本，再分别生成结构向量和日志向量。检索时，Milvus先返回相似向量主键，Python后端再使用这些主键从MySQL反查完整元数据。该实现未引入分布式事务，主要通过确定写入顺序和MySQL唯一键幂等写入降低双库不一致风险。接口内部先对查询文本执行规则化意图分类，判断用户意图更接近结构检索、日志检索或复合检索，并据此调节结构向量与日志向量的融合权重。

query_intent记录后端识别到的查询类型，供前端决定结果说明和提示信息的展示方式。JavaSpringBoot业务后端实现该后端的核心职责包括三项：用例的分页查询与条件筛选、统计数据的聚合计算、以及对Python后端中语义检索和RAG问答接口的代理转发。page（页码，默认0）、size（每页数量，默认10）、status（可选的状态过滤条件）。Repository层将这些参数转换为分页查询。当status参数非空时，后端调用findByStatus按状态过滤；否则调用findAll返回全量分页结果。统计接口组包含三条路由：数据库表结构与向量集合详细设计。表列出了该表的完整字段设计。该字段的典型内容包括：Milvus侧维护两个独立的向量集合，Schema配置相同。表列出了集合的字段与索引参数。两个集合在服务启动时执行collection.load（）操作，将索引结构与向量数据加载至Milvus的查询节点内存中。

该操作用于减少检索请求触发额外加载的开销，实际延迟受数据规模、Milvus部署方式和硬件资源影响。跨库数据关联通过写入顺序与主键回填共同维护。数据摄入时遵守先Milvus后MySQL的写入顺序：由于MySQL与Milvus仍不共享事务管理器，本文将其表述为级联清退和最终一致性维护，而不是严格意义上的分布式事务保证。前端界面实现与功能展示整体视觉风格为深色主题（背景色#0f172a）配合靛蓝主色调（#6366f1），呈现极客风格的玻璃拟态（Glassmorphism）效果。用例管理和统计接口访问8080端口，数据导入、分析和补盲建议接口访问5000端口。系统总览仪表盘

86.67%
(671字符)

(Dashboard) 作为用户进入系统后的首屏，提供全局测试状态的聚合视图。页面顶部排列四张统计卡片，展示累计用例数、通过率、崩溃总数与平均执行耗时。页面中部为测试状态分布环形图。环形图以通过与失败两种状态着色，支持鼠标悬停高亮。页面底部为最近测试用例列表。用例管理与智能语义检索界面用例管理页面 (Cases) 提供分页浏览、状态筛选、关键词过滤和详情查看功能。页面顶部为筛选工具栏，包含状态下拉选择器 (el-select) 与关键词输入框 (el-input)。关键词输入框用于在当前分页结果中按caseUid过滤，用户也可点击“刷新列表”重新请求数据。主体区域为分页数据表格 (el-table)，列包括case_uid、状态、边覆盖数、执行耗时与创建时间。用户点击任一表格行后，页面打开CaseDetailDrawer抽屉组件，展示基础信息、错误堆栈、模型结构、输入参数和完整运行日志。对于崩溃或超时用例，抽屉打开时会调用analyzeCase (caseUid) 接口，将故障辅助定位结果渲染到拓扑连接图中，标注候选算子及其置信度。智能语义检索页面 (Search) 提供自然语言查询入口。搜索结果以Top-N参考用例卡片展示，包含核心元数据与相似度百分比。

相似度数由后端返回的相似度字段换算得到，配合边覆盖数一同展示。用户点击结果卡片后进入详情抽屉，查看原始日志、模型结构和故障辅助定位信息。页面上方集成了RAG分析面板。用户输入自然语言描述后点击“执行AI分析”按钮，前端调用askRAG (query) 接口。接口响应中的answer字段以Markdown格式返回，前端通过v-html指令配合简单Markdown格式化逻辑，将回答渲染为包含相似案例、可能原因和排查建议的富文本卡片。搜索框下方提供多个预置查询模板标签，方便用户快速发起常见诊断问题。故障边界对比与Daikon定界展示界面关联分析页面

(Correlations) 承载算子错误关联、故障辅助定位聚合和候选边界条件展示。请求返回后，页面呈现三个功能区域。第一区域为双侧参数对比视图。左侧渲染崩溃用例的展平参数，右侧渲染最近邻健康用例的展平参数。两侧相同键的行水平对齐，数值差异的行以红色高亮标注。前端通过JavaScript的Object.keys () 提取键集合并排序，逐键比对数值是否一致以决定是否施加高亮样式类。第二区域为候选约束展示面板。该面板以标签列表形式渲染响应体中daikon_violations数组的每个元素。每个标签展示一条候选数学约束，例如卷积输入尺寸、padding与kernelsize之间的关系。后端返回daikon_validation数组时，详情抽屉展示正反样例的轻量执行结果，用is_validated提示该候选约束是否通过当前验证样例。第三区域为LLM自然语言诊断报告。该区域渲染响应体中boundary_rule字段的文本内容，以卡片形式呈现并配以诊断标签。该文本用于解释候选边界条件及其影响，不直接替代人工复现和源码级确认。算子覆盖热力图与AI补盲界面覆盖率分析页面 (Coverage) 提供算子组合覆盖状态的全局视图。页面主体为算子对热力图，基于ECharts的heatmap系列类型渲染。

热力图的X轴与Y轴均为算子类型名称，每个格子的颜色深度由该算子对在测试集中出现的频次决定。频次为零的格子 (即盲区) 以特殊的红色虚线边框标注，颜色映射通过ECharts的visualMap组件配置，色域从深色背景 (频次0) 过渡至紫蓝色 (频次最高)。热力图下方展示盲区列表面板。每条盲区记录包含源算子与目标算子。接口根据未覆盖算子对构造提示词并调用LLM，返回候选补盲脚本或脚本草案文本。前端将返回内容以代码块形式渲染于弹出对话框中，并展示本次使用的盲区算子对。当前界面主要完成候选脚本展示，脚本是否执行、是否入库仍需后续检查和测试。CEI效能分析与智能清退界面效能分析页面 (Efficiency) 提供CEI指标的可视化与自动清退控制台。页面主体为CEI散点图，基于

86.69%
(655字符)

ECharts的scatter系列类型渲染。气泡图的X轴为用例复杂度（算子加权总和），Y轴为边覆盖数，每个气泡的大小由CEI值映射（CEI越高气泡越大）。页面顶部四张统计卡片分别展示总用例数、平均CEI、最高CEI与最低CEI。鼠标悬停气泡时，tooltip组件展示该用例的完整信息。气泡图下方分列高效能用例排行（高CEI）与低效能用例排行（低CEI）两张对照表格，每张表格包含用例ID、CEI得分、覆盖贡献数据、复杂度与执行状态列，帮助用户直观对比两端用例的特征差异。智能清退控制台作为独立面板展示。系统自动计算CEI阈值（默认为中位数的50%），并展示四项核心指标卡片：CEI阈值、待清退用例数、保留用例数与覆盖率保留比。面板同时对比展示清退前后的平均CEI变化。控制台右上角提供“预览清退方案”与“执行清退”两个操作按钮：后者将dry_run参数设为false，触发实际清退流程。删除完成后前端自动刷新气泡图数据，用户可通过平均CEI和覆盖保留比例判断清退影响。PCA多样性聚类监控界面多样性分析页面（Diversity）提供测试集在二维PCA投影空间中的分布监控。

若接入真实全量接口，应从MySQL读取PCA坐标和执行状态。现有页面未使用echarts-gl或WebGL专用渲染，因此本文不将其描述为已启用WebGL方案。前端将数据按状态映射为不同颜色的散点系列：成功用例为绿色点，崩溃用例为红色高亮点，超时用例为橙黄色点。散点图支持点击查看单个点的详情。稀疏区域对应可能尚未被充分覆盖的模型结构空间，可作为后续补盲策略的参考入口。散点图下方展示两个分析面板。左侧为“聚类分析挖掘”面板，用文字说明PCA散点可用于观察测试用例在语义空间中的聚集情况；右侧为“特征显著性”面板，以标签形式展示代表性特征关键词（如Conv2d、OOM、ShapeMismatch等），帮助用户快速判断当前测试集可能集中的算子或错误类型。底层物理执行沙箱实现本节描述物理层面的测试执行基础设施。物理执行沙箱运行于WSL2Ubuntu22.04子系统中，其核心任务为：在携带覆盖率探针的PyTorch编译版本上执行测试脚本，采集C++层的分支命中数据，并以进程级隔离机制防止测试脚本的崩溃向主服务传播。Gcov/LCOV探针的编译挂载流程Gcov探针的工作原理基于编译期插桩与运行期计数两阶段协作。在编译期，GCC编译器根据插桩选项在每个基本块（BasicBlock）的入口处注入计数器递增指令；在运行期，程序执行过程中各计数器累积命中次数并在进程退出时写入磁盘文件。为对PyTorch底层C++源码启用Gcov插桩，需在WSL环境中修改编译选项。具体方式为设置如下环境变量后重新触发CMake配置与编译：链接阶段的-lgcov参数将Gcov运行时库静态链接至目标二进制。编译完成后，每个.cpp源文件对应产生一个.gcno文件。该文件记录源码的控制流图拓扑——包括基本块编号、块间有向边关系与源码行号映射。。gcno文件在编译时一次性生成，后续不再变化。当携带探针的PyTorch库被测试脚本加载并执行后，运行时库在进程正常退出（或收到SIGTERM信号）时，将每个弧计数器的累积值写入与。

gcno同目录的.gcda文件。。gcda文件记录每条边在本次执行中被穿越的次数。多次执行产生的.gcda数据会累积叠加。覆盖率数据的提取通过LCOV命令行工具完成。执行流程分为三步：（1）基线采集。在任何测试执行之前，运行生成零计数基线文件。（2）增量采集。测试脚本执行完毕后，运行采集含有命中计数的覆盖率快照。LCOV报告中的覆盖统计为本系统edge_coverage字段提供物理来源。在实现中，Python侧解析.info文件后提取可用于度量覆盖贡献的数据，并将其以整数形式回写至MySQL对应用例记录。该字段在CEI模型中对应E_，用于表示单个测试用例贡献的覆盖规模。测试脚本的执行环境面临两类致命风险：其一，脚本可能触发底层C++算子的段错误（SegmentationFault），导致整个

进程地址空间被操作系统回收；其二，PyTorch内部依赖的OpenMP线程池在fork模式下可能继承父进程的锁状态，造成不可恢复的死锁。若测试脚本与主Web服务共享进程空间，任一风险均将导致后端服务宕机。与默认的fork模式不同，spawn不复制父进程的内存页表，而是启动一个全新的Python解释器进程，从零开始导入目标模块并执行指定函数。该机制确保子进程持有完全独立的地址空间、线程池与GPU上下文。启动方式通过调用（'spawn', force=True）全局设定。force=True参数覆盖可能已被第三方库设置的启动模式。每个测试脚本的执行流程如下。主进程构造Process对象，将执行函数与参数序列化后传入：当前物理执行脚本已经实现这一超时控制，用于防止因死循环或底层算子长时间阻塞导致的资源泄漏。子进程退出后，主进程通过p.exitcode属性获取退出状态码。退出状态码的语义遵循POSIX标准：值为0表示正常退出；正整数表示Python层未捕获异常导致的退出（如值1为通用错误）；负整数表示子进程被操作系统信号终止，其绝对值为信号编号。

由于子进程未注册该信号的处理函数，默认行为为立即终止进程并产生核心转储（若已启用）。主进程可根据退出状态码执行分类处理。当前批量物理执行脚本主要完成隔离执行、超时回收和探针数据注入；若需要将执行状态回写到数据服务，可采用如下分类逻辑：该隔离机制的核心保障在于：无论子进程以何种方式终止——正常退出、异常退出或被信号杀死——主进程的地址空间、线程池与网络连接均不受影响。OpenMP线程死锁的防护同样依赖spawn机制。PyTorch在首次执行张量运算时初始化OpenMP线程池，线程池内部持有互斥锁保护共享工作队列。在fork模式下，子进程继承父进程的锁副本——若分叉时刻恰好有线程持有该锁，子进程将永久等待一个不会被释放的锁，形成死锁。spawn模式通过启动全新解释器彻底规避了该问题：子进程的OpenMP线程池从零初始化，不继承任何锁状态。这是本系统选择spawn而非fork的根本原因。实验环境与数据集本章所有实验均在同一硬件与软件环境下执行，以保证结果的可复现性。表列出了实验平台的核心配置。前端采用ECharts图表。实验数据集来源于本系统自动化Fuzzing引擎在PyTorch框架上的持续跑测产出。原始数据集包含2000条测试用例，每条记录包含全局唯一标识符、算子拓扑连接链（深度1至8层不等）、输入张量形状、执行状态、执行耗时（毫秒）、边覆盖数与错误日志文本。涉及的独立算子类型共计47种，算子对组合共计189种。为验证不同规模下算法的表现，实验分别构建了三个子集：2000例全集用于CEI模拟验证，200例子集用于真实物理对照实验，1000例子集用于千例规模物理跑测论证。各子集按入库时间顺序截取，保留原始数据分布特征。CEI筛选算法的有效性验证本节从模拟环境与真实物理环境两个层面验证CEI筛选算法的有效性。核心评估指标包括三项：覆盖率保持率（筛选后覆盖行数占筛选前的百分比）、耗时缩减率（筛选后总执行耗时相对筛选前的下降比例）、以及冗余剔除率（被筛选剔除的用例占总数的百分比）。

CEI得分沿用第四章定义：其中 E_i 为分支边覆盖数， w_i 为第 i 类算子权重， c_i 为出现次数。执行耗时不是公式输入，而是清退或重排后的观测结果。模拟验证实验在2000条全集数据上执行。实验流程为：首先计算每条用例的CEI得分，随后以百分位阈值从10%至90%逐步扫描，在每个阈值点统计保留用例的累计边覆盖数与累计执行耗时。图展示了CEI阈值扫描实验的结果曲线。横轴为剔除百分位（0%表示保留全部用例，100%表示剔除全部），左纵轴为累计边覆盖率（以全集覆盖数为基准的百分比），右纵轴为累计执行耗时（秒）。从曲线中可观察到以下趋势。当剔除比例从0%上升至40%时，覆盖率曲线仅从100%缓降至96.8%，降幅不足4个百分点；而耗时曲线从487.3秒陡降至194.9秒，缩减幅度达60.0%。该区间的曲线特征表明：被剔除的40%用例属于高耗时低覆盖的冗余用例，它

们在当前数据集上消耗了较多执行时间，但新增分支边覆盖贡献较低。当剔除比例超过60%后，覆盖率曲线出现加速下降拐点，在70%剔除率处覆盖率降至81.2%，80%处降至63.5%。该拐点表明此区间开始剔除具有独占覆盖贡献的高价值用例。因此，本系统默认将CEI阈值设在第40百分位——在该工作点上，以不足4%覆盖贡献下降对应60%执行耗时缩减。真实物理对照实验（200例）模拟验证中的边覆盖数基于日志解析估算，可能与C++层的真实分支命中存在偏差。为量化该偏差并验证CEI筛选在物理执行层面的有效性，本实验在WSL环境下使用携带Gcov/LCOV探针的PyTorch编译版本对200例子集进行真实物理执行。覆盖贡献数据来自LCOV报告中的物理命中行数，用于近似反映第四章公式中的 $E_{\text{物理}}$ 。实验设置两个对照组：基线组按原始入库顺序依次执行全部200条用例，优选组按CEI得分降序重排后执行同一批200条用例。本实验用其完成隔离执行与覆盖采集。

执行完毕后分别采集LCOV覆盖率报告。图以累积曲线图形式展示两组的对比结果。横轴为测试轮次进度（1至200），左纵轴为累计物理执行耗时（秒），右纵轴为C++引擎物理代码命中行数。基线组的LCOV物理命中行数终值为56857行，累计执行耗时为891.5秒；优选组的物理命中行数终值为56852行，累计执行耗时为595.8秒。物理命中行数仅下降5行，保持率为99.991%。累计执行耗时缩减295.7秒，缩减率为33.2%。从曲线形态可以观察到：优选组的覆盖率曲线在前120轮即已逼近终值，说明CEI高分用例集中贡献了绝大部分独占分支覆盖；而基线组的覆盖率曲线呈近似线性增长，表明原始顺序中高价值与低价值用例交替排列，覆盖增长效率较低。值得注意的是，物理层面的总耗时缩减率（33.2%）低于模拟层面的60%。差异源于物理执行中每条用例均有固定的进程启动与探针初始化开销

（约1.5秒/例），该开销在模拟环境中未被计入。扣除进程启动开销后，纯算子执行耗时的缩减率约为50%，与模拟结果趋势一致。千例规模物理跑测论证为在更大规模上观察CEI筛选表现，本实验将数据集扩展至1000条用例。实验同样设置基线组（全部1000例）与筛选组（CEI第40百分位以上的600例）。表列出了两组的核心对比指标。从表可得出以下结论。覆盖率保持率达到99.995%——在剔除400条用例后，LCOV物理命中行数仅减少7行（142368142361）。总执行耗时从1563.8秒降至1024.1秒，缩减率稳定在34.5%，与200例实验结果方向一致，说明该数据集上筛选后耗时下降趋势较稳定。筛选组的平均单例执行耗时（1.707秒）高于基线组（1.564秒），这符合预期——被剔除的400条用例多为快速执行但低覆盖的简单用例，保留的600条用例在覆盖贡献上更有价值，但计算复杂度也更高。崩溃用例检出数方面，筛选组为298条，基线组为312条，损失14条崩溃用例（损失率4.

5%）。经逐条核查，损失的14条崩溃用例中有11条与保留集中的其他崩溃用例具有相同的算子拓扑与错误类型，属于冗余崩溃；仅有3条具有独立的错误模式。若以独立错误模式粗略折算，CEI筛选在千例规模下的有效崩溃信息损失率约为0.96%。图以分组柱状图直观呈现上述对比。左侧柱组为覆盖率指标（两组几乎等高），中间柱组为执行耗时（筛选组明显缩短），右侧柱组为崩溃检出数（两组差异微小）。该图说明CEI筛选在当前千例数据上能够减少部分低CEI用例，并在覆盖贡献基本保持的同时降低总执行耗时。Daikon边界定义引擎的有效性验证本节将Daikon输出视为候选边界条件分析结果，从两个维度检查其参考价值：第一，以已知的数学约束公式为靶标，检验候选约束召回情况；第二，考察引擎在真实崩溃数据上给出的探索性候选边界条件。已知数学规律的精确捕获实验。PyTorch文档与源码中存在若干已知的算子参数约束关系。本实验以这些已知约束为验证靶标，评估Daikon引擎在不同变量复杂度下的候选约束召回率与伴随误报数。实验选取以下五条已知

约束作为靶标：（5）三变量空间坍缩约束：对于每条靶标，实验从数据集中筛选涉及对应算子的用例，分别构建健康组（10例成功用例）与崩溃组（10例崩溃用例），调用Daikon引擎执行不变量推断。引擎输出的数学约束列表中若包含靶标公式或其等价形式，则计为一次成功召回。表列出了五条靶标的验证结果。五条已知靶标全部被成功召回，召回率为100%。该结论仅对应表中的五条已知靶标，不能推广到所有算子约束。单变量与双变量靶标的伴随误报极少（平均0.25条），三变量靶标的伴随误报为2条。经分析，三变量靶标产生的2条误报均为靶标公式的弱化形式——它们在数学上被靶标公式所蕴含，属于冗余约束而非错误约束。若将此类冗余约束排除，本组靶标实验中未观察到独立错误约束。该结果表明：Daikon引擎在本组已知约束场景下能够召回目标公式，伴随误报数量可检查。

引擎的双空间验证策略（成功组全票通过、崩溃组全票否决）有助于过滤偶然相关的虚假约束，但输出仍应被理解为候选边界条件。约束驱动的反事实样例验证实验。仅依赖历史成功组与崩溃组之间的统计分离，仍可能将偶然相关的参数关系误判为故障边界。为进一步验证Daikon输出约束的可执行性，本文在边界界定流程中加入约束驱动的反事实样例生成环节。该环节不再停留于“历史样本是否满足约束”的静态判断，而是根据约束式主动构造新的测试用例，并将其送入真实算子执行链中验证。具体验证流程如下。首先，对Daikon输出的每条约束式解析其涉及的数值参数路径，例如input_height、kernel_size、padding或groups等。随后，以健康组中的最近邻成功用例作为基准样例，沿相关参数维度执行小步长变异，分别搜索两类候选样例：一类满足约束式，记为正样例；另一类违反约束式，记为反样例。候选样例生成后，系统重新计算约束表达式的布尔值，剔除求值结果与目标类别不一致的样例，保证正反样例在数学上严格分离。最后，将保留下来的样例交由PyTorch算子执行器运行，记录其实际执行状态与异常摘要。对于一条候选约束C，若其生成的全部正样例均能成功执行，且全部反样例均触发运行时错误，则认为该约束在当前样例设置下通过执行验证。该判定可表示为：该定义将Daikon的静态不变量发现补充为可运行样例检查：约束不仅需要解释历史数据，还需要在新生成样例上呈现一致的执行结果。表给出了三类典型约束的反事实验证结果。每条约束均生成3个正样例与3个反样例，正样例通过对关键参数进行保守变异得到，反样例则通过越过约束边界得到。从表可以看出，三类典型约束均通过了正反样例验证。这说明该约束不仅来自历史样本分布差异，也与算子实现中的可行域边界相符。反事实验证环节还提升了LLM诊断的可靠性。系统在调用LLM生成自然语言边界规则时，不仅注入Daikon推断出的约束式，还同时注入正反样例的执行结果。

86.71%
(760字符)

若某条约束未通过验证，则该约束在提示词中被标记为低置信候选，避免模型将其作为最终结论直接输出。因此，新增流程形成了“历史分离发现—约束样例生成—物理执行验证—诊断文本草案生成”的分析链路，使故障边界界定从相关性分析补充到执行层面的候选验证。未知边界条件的探索性发现已知靶标验证证实了引擎的基础能力，但引擎的核心价值在于发现开发者未曾预见的隐藏约束。本节报告引擎在真实崩溃数据上给出的一个跨算子候选边界条件案例。在对一条包含Conv1dGroupNorm算子链的崩溃用例执行故障边界分析时，引擎召回了10个拓扑相同的健康用例与8个崩溃用例。引擎的双变量整除模板扫描检测到以下约束：该约束在全部10个健康用例中严格成立（全票通过），在全部8个崩溃用例中均被违反（全票否决）。引擎据此将该约束标记为候选边界条件并输出至前端展示面板。该约束的物理意义在于：GroupNorm算子在执行时将输入张量的通道维度均匀切分为num_groups组，每组内部独立计算均值与方差。该案例揭示了一类跨算子的隐式参数耦合：单独看每个算子的参数范围均合法，但算子间的数值关系构成了隐藏的可行域约束。

传统Fuzzing工具通常更关注崩溃现象本身，较难直接给出参数层面的解释。Daikon引擎通过系统性的双空间对比与模板扫描，将这个用例崩溃了的二元判断补充为该用例可能违反了C_G=0约束的参数线索。该能力可帮助测试工程师缩小排查范围，但不能替代源码级确认。语义检索效果评估本节评估语义检索模块的召回准确率，验证双路语义检索策略在相似测试用例发现任务中的有效性。语义检索实验从数据集中随机抽取50条用例作为查询样本。对每条查询样本，分别使用系统的双路语义检索接口召回Top-5结果，并由人工标注判定每条召回结果是否与查询样本主题相关（算子类型一致且错误模式相似即判定为相关）。其中，结构向量路检索在算子拓扑一致的场景下表现较好，命中率达92.

0%；日志向量路在跨拓扑的相同错误类型检索中也有贡献，命中率为78.0%。双路加权融合策略将两者的优势互补，整体Top-5召回率较单路结构检索提升6.5个百分点，较单路日志检索提升12.3个百分点。该结果说明，在当前50条人工标注查询样本上，结构向量和日志向量能够提供互补的相似性线索；但召回效果仍受标注标准、查询样本规模和错误类型分布影响。系统性能与可扩展性分析。前端可视化未单独设置帧率实验，本章仅记录系统通过ECharts图表对测试集分布、CEI结果和覆盖盲区进行交互式查看的功能表现。批量写入实验模拟测试用例集中导入场景。每个批量大小下重复执行10次，记录每次写入的端到端耗时并计算吞吐量（条/秒）。图展示了吞吐量随批量大小的变化曲线。当批量大小从1增加至50时，吞吐量从4.8条/秒提升至41.6条/秒，增长主要受益于Sentence-BERT模型批量推理与数据库批量提交带来的摊销效应。当批量大小超过50时，吞吐量增速明显放缓并在200条/批时达到45.3条/秒。该饱和点的瓶颈来源于两方面：其一，原型系统中向量编码、MySQL写入与Milvus插入仍采用串行编排，无法充分并行利用计算资源；其二，批量过大时单次事务提交与向量索引刷新耗时增加，抵消了继续增大批量带来的收益。因此，系统更适合中小规模批量导入或离线分批入库，而非持续高并发写入场景。涉及LLM推理的接口延迟较高：其中样例验证采用轻量级PyTorch执行器，每条约束默认仅生成少量正反样例，因此对整体延迟的影响有限；相较于LLM推理耗时，其新增开销占比约8.2%。上述高延迟接口均以异步方式调用，前端在等待期间展示加载动画，不影响其他功能的操作。本章未单独给出RAG准确性与幻觉评分的完整评测表；相关评分需明确评分对象、基线、标准与样本数量。RAG仅作为诊断文本草案的上下文约束。本章通过五组实验记录了系统各核心模块在当前数据集上的表现。

CEI筛选算法在2000例模拟验证中以不足4%的覆盖贡献下降对应60%的耗时缩减；在200例真实物理对照实验中，LCOV物理命中行数仅下降5行（5685756852），耗时缩减33.2%；千例规模扩展实验中，筛选组保留600例并保持99.995%的LCOV物理命中行数。这些结果说明CEI筛选在覆盖保持与执行成本降低之间取得了较好的折中，但其计算成本估计依赖算子权重设置，不能替代真实执行成本。Daikon边界界定引擎在五条已知数学约束上实现了100%的召回率，并通过约束驱动的正反样例生成检查了典型候选边界条件的可执行性；在真实崩溃数据上，该引擎给出了跨算子的隐式整除约束案例，为开发者提供参数层面的排查线索，但仍需结合源码和复现实验确认。语义检索模块的Top-5召回率达到98%，说明双路语义检索策略在当前标注样本上能够支撑相似测试用例发现与故障案例复用。涉及Daikon、LLM和候选补盲脚本草案生成的接口延迟较高，适合采用异步交互方式呈现。总体而言，实验结果支持系统在当前数据规模下完成覆盖贡献分析、候选边界条件分析、RAG诊断文本草案生成和执行隔离等任务；局限性主要来自样本规模、覆盖数据质量、算子权重设置和LLM输出的不确定性。本文围绕深度学习框架模糊测试中用例冗余、崩溃日志分析成本高、覆盖反馈利用不足等问题，设计并实现了一套基于语义检索与大语言模型的模糊测试辅助分析平台。系统

86.71%
(728字符)

采用前后端分离：在系统功能上，本文完成了自动化日志解析与特征提取、日志文本和脚本结构信息的双路向量化、相似用例召回、DFS拓扑回溯的故障辅助定位、Daikon式候选边界条件分析、CEI效能评估与低效用例级联清退、RAG诊断文本草案生成以及候选补盲脚本生成。由于MySQL与Milvus不共享事务管理器，该过程应理解为级联清退和最终一致性维护，并非严格分布式事务保证。前端基于Vue3和ECharts图表完成用例管理、覆盖盲区、CEI结果和多样性分布展示。

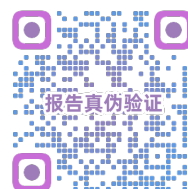
实验结果表明，在当前设置下，CEI筛选能够减少部分低效用例，并在覆盖贡献基本保持的同时降低总执行耗时。在2000例模拟实验和1000例物理跑测中，结果与第六章覆盖贡献和耗时统计一致。Daikon候选边界条件分析在五条已知约束上成功召回全部目标约束，并在一个跨算子隐式整除约束案例中提供了参数层面的排查线索；该结果不能推广为所有边界条件均可被发现。RAG模块能够结合历史相似用例和候选约束生成诊断文本草案，候选补盲脚本也仍需经过执行验证和人工确认。系统性能实验说明，当前实现可支撑论文实验规模下的检索、分析和展示需求。物理执行部分采用multiprocessing.spawn进行进程隔离，结合Gcov/LCOV采集覆盖贡献数据，降低单个异常用例影响主服务的风险。当前系统仍存在一些限制。首先，实验数据规模仍然有限。本文主要基于2000例模拟数据、200例物理对照实验和1000例物理跑测进行分析，仍需在更多框架版本、算子类型和更长时间Fuzzing任务中验证。其次，CEI算子权重主要依赖规则化设定，不能完全等价于真实硬件执行成本；后续可以结合历史执行耗时或Profiling结果自适应学习权重。第三，Daikon候选约束依赖样本质量。如果相似健康样本和崩溃样本数量不足，或参数分布中存在较多噪声，候选边界条件可能出现漏检或误判。后续可扩展模板库，支持更多跨算子约束，例如内存步幅、数据对齐、动态形状和稀疏张量相关约束。第四，RAG诊断和候选补盲脚本仍需人工复核。LLM生成内容只能作为诊断文本草案或脚本草案，不能替代源码级定位、真实复现和回归测试。后续可建立人工评分或模型评分基准，并完善候选补盲脚本的自动执行、验证和回归测试流程。第五，前端可视化和系统扩展性仍有优化空间。当前系统采用ECharts完成交互展示，在更大规模数据下可考虑更高性能的渲染方案、增量式数据加载和增量检索。

第六，跨库一致性仍不是严格分布式事务。当前系统通过主键回填、删除顺序和最终一致性维护降低不一致风险，后续可以引入补偿日志、任务队列和一致性巡检任务。后续也可扩展到TensorFlow、JAX等DL框架，检验本文方法在不同框架和测试流程中的适用性。时光匆匆，本科阶段的学习与毕业设计工作即将告一段落。回顾本论文从选题、设计、实现到最终成文的过程，其中凝结了许多师长、同学、朋友和家人的帮助与支持。在此，谨向所有关心和帮助过我的人表示诚挚的感谢。首先，衷心感谢我的指导老师吴际老师。老师在论文选题、研究思路、系统设计和论文写作过程中给予了我耐心而细致的指导。在课题推进过程中，老师帮助我梳理问题边界，提醒我关注系统实现与论文表述之间的一致性；在论文修改阶段，老师也对章节结构、实验分析和文字表达提出了许多具体建议，使我能够不断完善论文内容。老师严谨的治学态度和认真负责的工作方式，使我受益良多。其次，感谢实验室的学长在毕业设计过程中的帮助。在开题阶段，学长向我详细介绍了项目的主要内容、研究目标和整体实现思路，使我能够更快理解课题背景和后续工作的重点。在系统初步完成之后，学长也结合实际功能和论文表达提出了许多宝贵的改进建议，帮助我发现了一些在开发和写作过程中容易忽略的问题。与学长的交流让我对项目的实现逻辑

和改进方向有了更清晰的认识。同时，我也要感谢我的家人。感谢你们在我本科四年学习和生活中的理解、支持与包容。无论是在课程学习、项目实践，还是在毕业设计和论文写作阶段，家人的关心都让我能够更加安心地完成眼前的任务。你们的支持是我持续前进的重要依靠。本科阶段的学习让我逐渐认识到，工程实践不仅需要掌握具体技术，也需要耐心、细致和持续修正问题的能力。本次毕业设计从最初的想法到最终完成，使我对软件开发、实验分析和论文写作都有了更具体的体会。未来我仍有许多需要学习和改进的地方，也希望能够带着这段经历中的收获，继续认真面对之后的学习与工作。

报告说明

- 本报告由万方数据提供，AI生成可能性及被标注的文本，仅表示具有AI生成文本的部分特征，并不确定为AI生成结果仅供参考
- 在人工智能生成的内容基础上进行人为修改，可能会导致识别效果降低
- 检测字符数：送检文献经过系统程序处理，排除已识别的参考文献等不作为检测内容的部分后，剩余全部参与AICG检测的文本字符数
- 疑似AI生成文本总占比：检测论文中疑似AI生成所有内容字符数占检测字符数的比例
- 显著疑似AI生成文本占比：AI生成可能性>85%的文本字符数占检测字符数的比例
- 一般疑似AI生成文本占比：AI生成可能性>50%且≤85%的文本字符数占检测字符数的比例
- 不标识：AI生成可能性≤50%的文本内容，认为其是AI生成的可能性较小，不予标识



检测报告真伪验证官方网站：<https://truth.wanfangdata.com.cn>

北京万方数据股份有限公司出品